



UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN



FIME

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

“Tarea Práctica #2: Hilos (Individual)”

Sistemas Distribuidos y Paralelos

M.I.I. Carlos Adrián Pérez Cortez

Grupo: 004

Hora: N5

Salón: 4207

MATRICULA	NOMBRE	PE
1952947	Heidi Pamela Martínez Martínez	ITS

Ciudad Universitaria, San Nicolás de los Garza, Nuevo León a 27 de agosto
de 2024

CAPTURA DE PANTALLA DEL CÓDIGO EN JAVA

NUMALEATORIOS

```
import java.io.FileWriter;
import java.io.IOException;
import java.util.Random;

public class NUMALEATORIOS {
    Run | Debug
    public static void main(String[] args) {
        int cantidadNumeros = 100000; // numeros
        String archivo = "numeros.txt";

        try (FileWriter writer = new FileWriter(archivo)) {
            Random random = new Random();
            for (int i = 0; i < cantidadNumeros; i++) {
                int numero = random.nextInt(bound:1000); // Números aleatorios entre 0 y 999
                writer.write(numero + (i < cantidadNumeros - 1 ? ", " : ""));
            }
            System.out.println(x:"Archivo listo.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

ARCHIVO DE TEXTO

```
Archivo  Editar  Ver

481,12,551,669,351,469,695,533,917,755,401,7,92,729,41,826,153,410,508,731,411,508,487,412,568,551,384,116,79,593,54,735,121,591,725,
858,313,994,73,429,650,67,228,205,584,252,698,149,462,421,825,304,367,712,7,518,440,801,435,33,2,565,805,234,501,446,994,469,472,561,
456,470,134,269,376,990,386,399,729,374,35,966,896,182,170,736,69,611,72,31,890,499,509,52,325,870,27,934,260,707,401,271,984,163,427
,140,169,134,647,166,857,913,138,594,390,457,257,513,447,983,100,340,647,327,916,457,807,694,45,394,654,63,759,800,300,139,87,876,748
,123,34,925,704,211,977,331,406,220,279,967,663,390,0,541,752,969,84,111,10,886,81,527,189,767,967,904,617,135,180,498,60,95,344,640,
608,182,450,696,460,708,233,670,386,708,256,189,969,972,217,283,961,858,298,296,867,252,612,267,595,252,120,249,662,263,988,59,501,19
2,299,603,940,957,643,698,676,869,240,281,780,525,494,370,687,494,3,498,14,117,851,293,654,911,805,406,540,214,891,242,893,668,495,39
2,707,923,536,343,689,774,796,859,439,152,687,222,265,479,274,207,122,204,487,370,230,642,233,132,380,633,856,321,435,478,130,811,756
,649,477,617,828,482,16,754,437,848,794,252,419,944,674,542,561,838,341,894,630,667,118,95,570,668,752,763,658,538,903,113,619,123,38
9,19,115,677,128,407,675,314,168,728,736,724,447,0,761,7,409,764,483,877,42,892,56,418,223,150,943,829,700,997,180,15,141,691,408,467
,576,952,403,162,863,438,310,417,452,883,245,266,846,530,321,732,384,358,4,608,287,300,357,753,60,652,804,447,752,939,88,456,268,433,
51,851,523,250,119,507,631,475,601,204,152,58,572,381,707,154,241,137,399,546,618,36,611,545,430,471,995,633,478,745,201,45,750,730,1
27,447,136,292,987,299,498,88,124,266,983,690,881,995,563,115,33,708,32,777,469,889,726,357,633,135,656,256,713,456,521,394,704,795,8
04,73,788,551,923,299,470,908,696,929,125,934,71,472,218,694,268,748,765,967,689,739,871,810,776,643,769,327,623,81,130,193,609,359,3
58,308,48,17,729,485,561,101,667,750,472,708,469,972,234,33,89,207,594,829,552,734,547,221,406,557,508,505,247,60,777,161,85,501,798,
693,857,25,646,890,975,391,658,108,550,610,62,669,404,230,861,896,345,479,220,482,834,142,940,86,782,145,777,19,597,262,928,301,171,6
44,749,132,724,174,658,660,938,836,275,682,586,695,806,96,789,713,433,526,26,111,364,437,569,933,949,817,522,556,850,602,955,970,737,
191,32,86,803,984,981,814,527,966,567,15,770,465,681,632,100,237,475,660,199,10,820,362,225,623,880,122,224,91,428,878,437,103,734,35
,569,499,834,643,980,364,431,33,372,894,321,728,922,564,487,663,329,177,524,797,632,26,819,485,575,114,675,982,934,145,154,666,800,41
8,206,985,870,592,895,49,144,636,688,52,55,949,844,262,54,413,765,418,943,788,759,960,914,260,283,469,887,648,160,119,141,737,988,90,
459,502,33,572,939,495,186,450,407,630,44,893,515,267,586,397,754,3,160,560,573,988,297,190,614,591,708,596,443,179,177,516,45,810,61
354,195,562,393,329,222,954,688,700,498,63,355,876,73,325,335,146,925,317,780,867,845,611,535,98,666,348,953,18,136,106,980,990,505,8
60,453,656,339,537,741,234,154,151,585,388,504,624,946,202,952,740,207,329,130,923,654,791,856,830,369,76,823,701,853,544,626,726,246
282,57,703,335,379,878,304,701,677,104,320,939,523,664,290,832,70,361,603,316,624,813,467,210,409,746,214,983,636,363,174,57,574,835
,239,682,746,156,598,588,707,621,207,750,167,955,2,773,149,198,880,270,467,724,672,727,294,243,705,757,622,392,59,504,285,141,214,275
,725,526,998,392,907,572,34,424,364,315,32,304,50,830,486,287,986,578,447,420,926,624,897,196,162,308,303,337,837,843,512,446,374,297
,650,702,184,632,708,973,884,965,392,663,882,628,833,940,531,979,518,701,933,492,658,968,500,165,304,326,354,713,96,518,739,145,312,2
11,695,933,218,392,692,788,406,140,424,124,52,713,264,523,736,790,920,484,698,676,592,389,168,389,774,500,939,392,270,314,779,129,315
```

PROMEDIOHILOMAIN

```
1  import java.io.BufferedReader;
2  import java.io.FileReader;
3  import java.io.IOException;
4  import java.util.ArrayList;
5  import java.util.List;
6  import java.util.concurrent.Callable;
7  import java.util.concurrent.ExecutionException;
8  import java.util.concurrent.ExecutorService;
9  import java.util.concurrent.Executors;
10 import java.util.concurrent.Future;
11
12 class PromedioHilo implements Callable<Double> {
13     private List<Integer> numeros;
14
15     public PromedioHilo(List<Integer> numeros) {
16         this.numeros = numeros;
17     }
18
19     @Override
20     public Double call() {
21         int suma = 0;
22         for (int numero : numeros) {
23             suma += numero;
24         }
25         return (double) suma / numeros.size();
26     }
27 }
28
29 public class PromedioHiloMain {
30     Run | Debug
31     public static void main(String[] args) {
32         String archivo = "numeros.txt";
33         List<Integer> numeros = new ArrayList<>();
34
35         // Leer archivo y almacenar los números en una lista
36         try (BufferedReader br = new BufferedReader(new FileReader(archivo))) {
37             String linea;
38             while ((linea = br.readLine()) != null) {
39                 String[] valores = linea.split(regex:",");
40                 for (String valor : valores) {
41                     numeros.add(Integer.parseInt(valor));
42                 }
43             } catch (IOException e) {
44                 e.printStackTrace();
45             }
46
47             // Ejecucion sin hilos (lineal)
48             long inicioTiempoLineal = System.nanoTime();
49             double promedioLineal = calcularPromedioLineal(numeros);
50             long finTiempoLineal = System.nanoTime();
51             long tiempoLineal = finTiempoLineal - inicioTiempoLineal;
52             System.out.println("El promedio total Lineal es: " + promedioLineal);
53             System.out.println("Tiempo de ejecucion Lineal: " + tiempoLineal + " nanosegundos");
54
55             // Ejecucion con hilos (paralelo)
56             long inicioTiempoParalelo = System.nanoTime();
57             double promedioParalelo = calcularPromedioConHilos(numeros, cantidadHilos:4); // 4 hilos
58             long finTiempoParalelo = System.nanoTime();
59             long tiempoParalelo = finTiempoParalelo - inicioTiempoParalelo;
```

```

60     System.out.println("El promedio total Paralela es: " + promedioParalelo);
61     System.out.println("Tiempo de ejecucion Paralela: " + tiempoParalelo + " nanosegundos");
62 }
63
64 // Método para calcular el promedio de forma lineal
65 private static double calcularPromedioLineal(List<Integer> numeros) {
66     int suma = 0;
67     for (int numero : numeros) {
68         suma += numero;
69     }
70     return (double) suma / numeros.size();
71 }
72
73 // Método para calcular el promedio usando hilos
74 private static double calcularPromedioConHilos(List<Integer> numeros, int cantidadHilos) {
75     int totalNumeros = numeros.size();
76     int tamanoSegmento = totalNumeros / cantidadHilos;
77
78     ExecutorService executor = Executors.newFixedThreadPool(cantidadHilos);
79     List<Future<Double>> resultados = new ArrayList<>();
80
81     for (int i = 0; i < cantidadHilos; i++) {
82         int inicio = i * tamanoSegmento;
83         int fin = (i == cantidadHilos - 1) ? totalNumeros : inicio + tamanoSegmento;
84         PromedioHilo tarea = new PromedioHilo(numeros.subList(inicio, fin));
85         Future<Double> resultado = executor.submit(tarea);
86         resultados.add(resultado);
87     }
88
89     // Obtener los promedios y calcular el promedio total
90     double promedioTotal = 0.0;
91
92     try {
93         for (Future<Double> resultado : resultados) {
94             promedioTotal += resultado.get();
95         }
96     } catch (InterruptedException | ExecutionException e) {
97         e.printStackTrace();
98     }
99
100     // Cerrar el executor y obtener el promedio final
101     executor.shutdown();
102     promedioTotal /= cantidadHilos;
103
104     return promedioTotal;
105 }
106 }

```

CAPTURA DE PANTALLA DE LOS RESULTADOS OBTENIDOS EN CONSOLA

```
(base) C:\Users\heidi>cd Desktop  
(base) C:\Users\heidi\Desktop>javac NUMALEATORIOS.java  
(base) C:\Users\heidi\Desktop>java NUMALEATORIOS  
Archivo listo.  
(base) C:\Users\heidi\Desktop>javac PromedioHiloMain.java  
(base) C:\Users\heidi\Desktop>java PromedioHiloMain  
El promedio total Lineal es: 499.04694  
Tiempo de ejecucion Lineal: 6217900 nanosegundos  
El promedio total Paralela es: 499.04694  
Tiempo de ejecucion Paralela: 12456800 nanosegundos
```

CONCLUSIÓN

Conclusión

Heidi Pamela Martínez Martínez 1952947

La implementación en paralelo con hilos puede ofrecer mejoras significativas en el tiempo de ejecución, especialmente para las tareas que se pueden dividir y ejecutar en paralelo. Sin embargo, esto depende del tamaño de los datos y el número de hilos utilizados. En estos casos de datasets pequeños o cuando la creación de hilos es costosa, la versión lineal podría ser más eficiente. Por lo tanto, la decisión sobre cuando utilizar paralelismo debe basarse en el análisis de estos factores.

En resumen, si los resultados muestran una mejora en los tiempos con hilos, el paralelismo es ventajoso. Si no hay mejora, optimizar la implementación o reconsiderar el uso de hilos puede ser necesario.

BIBLIOGRAFÍA

Ricardo. (2022). Hilos (Thread) en JAVA. Programando En Java. Recuperado el 27 de agosto de 2024 del sitio web: <https://programandoenjava.com/hilos-thread-en-java/#:~:text=Hay%20dos%20formas%20de%20crear,Runnable>

Luraschi, G. (2023). Multi-hilos en Java. DEV Community; DEV Community. Recuperado el 27 de agosto de 2024 del sitio web: <https://dev.to/rlgino/multi-hilos-en-java-13pn>

Hilos en Java - clase Thread. (2024). Tutorialesprogramacionya.com. Recuperado el 27 de agosto de 2024 del sitio web: <https://www.tutorialesprogramacionya.com/javaya/detalleconcepto.php?codigo=180>

2022-02-09 Sist. Dist. y Paralelos V3 (2024). Sharepoint.com. Recuperado el 27 de agosto de 2024 del sitio web: https://uanledu.sharepoint.com/sites/Section_02316010133802031401000101843001/_layouts/15/stream.aspx?id=%2Fsites%2FSection%5F02316010133802031401000101843001%2FShared%20Documents%2FGeneral%2FRecordings%2F2022%2D02%2D09%20Sist%2E%20Dist%2E%20y%20Paralelos%20V3%2D20220209%5F134649%2DGrabaci%C3%B3n%20de%20la%20reuni%C3%B3n%2Emp4&nav=eyJyZWZlcjJhbnEluZm8iOnsicmVmZXJyYWxBcHAIoiJTdHJlYW1XZWJBcHAIcJyZWZlcjJhbnFZpZXciOiJTaGFyZURpYWxvZy1MaW5rliwicmVmZXJyYWxBcHBQbGF0Zm9ybSI6IldlYiIsInJlZmVycmFsTW9kZSI6InZpZXcifX0&ct=1724803504840&or=Teams%2DHL&ga=1&referrer=StreamWebApp%2EWeb&referrerScenario=AddressBarCopied%2Eview%2Eb5936323%2D2185%2D49f2%2Dbe24%2D943112082b34

Ormaza, J. (2020). ¡Hilos en Java! - Jesús Ormaza - Medium. Medium; Medium. Recuperado el 27 de agosto de 2024 del sitio web: <https://medium.com/@ormax563jj/hilos-en-java-919a49041da6>